

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ ĐÔNG Á
KHOA CÔNG NGHỆ THÔNG TIN



BÀI TẬP LỚN

HỌC PHẦN: TRÍ TUỆ NHÂN TẠO

BÁO CÁO: PHÂN LOẠI LOÀI HOA IRIS BẰNG HỌC MÁY

Giảng viên giảng dạy: Trần Xuân Thanh

Danh sách sinh viên thực hiện: Nhóm 10

STT	Mã sinh viên	Sinh viên thực hiện	Lớp hành chính
1	20232139	Nguyễn Thanh Hùng	DCCNTT.14.6
2			
3			

Bắc Ninh, năm 2026

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU BÀI TOÁN	3
1.1. Giới thiệu bài toán.....	3
1.2. Mô tả tập dữ liệu Iris	5
1.3. Mục tiêu.....	8
CHƯƠNG 2: PHƯƠNG PHÁP CHIA DỮ LIỆU (Hold-out)	10
2.1. Giới thiệu kỹ thuật Hold-out.....	10
2.2. Phương pháp chia dữ liệu Hold-out	12
CHƯƠNG 3: GIỚI THIỆU MÔ HÌNH VÀ THUẬT TOÁN.....	18
3.1. SVM (Support Vector Machine).....	18
3.2. Decision Tree.....	20
3.3. Naive Bayes	22
3.4. K-Nearest Neighbors (KNN)	24
CHƯƠNG 4: THỰC NGHIỆM VÀ KẾT QUẢ.....	26
4.1. Môi trường thực nghiệm	26
4.2. Kết quả từng mô hình (Accuracy, Precision, Recall, F1)	27
4.2.2. Mô hình Cây quyết định (Decision Tree)	29
4.4. Lựa chọn tham số tối ưu.....	38
CHƯƠNG 5: KẾT LUẬN	41
5.1. Kết luận chung.....	41
5.2. Đánh giá ưu điểm và hạn chế của đề tài.....	41
TÀI LIỆU THAM KHẢO	43

CHƯƠNG 1: GIỚI THIỆU BÀI TOÁN

1.1. Giới thiệu bài toán

Học máy (Machine Learning) là một lĩnh vực quan trọng của trí tuệ nhân tạo, tập trung vào việc xây dựng các phương pháp cho phép máy tính học từ dữ liệu và sử dụng những tri thức đã học để đưa ra dự đoán hoặc quyết định đối với các dữ liệu mới. Khác với cách tiếp cận truyền thống, trong đó người lập trình phải xây dựng trực tiếp các quy tắc xử lý cho từng trường hợp, học máy tìm cách phát hiện các quy luật tồn tại trong dữ liệu thông qua quá trình huấn luyện mô hình.

Trong số các bài toán cơ bản của học máy có giám sát (Supervised Learning), phân loại (Classification) là một dạng bài toán có vai trò đặc biệt quan trọng. Với bài toán phân loại, dữ liệu đầu vào đã được gán với các nhãn tương ứng và nhiệm vụ của mô hình là học mối quan hệ giữa các đặc trưng của dữ liệu với nhãn đó. Sau quá trình huấn luyện, mô hình được sử dụng để xác định lớp của những mẫu dữ liệu mới chưa xuất hiện trong tập huấn luyện.

Phân loại hoa Iris là một ví dụ kinh điển của bài toán phân loại trong học máy. Tập dữ liệu Iris có nguồn gốc từ công trình của Ronald A. Fisher công bố năm 1936 với tên *The Use of Multiple Measurements in Taxonomic Problems*. Trong nghiên cứu này, Fisher xem xét nhiều phép đo hình thái của hoa Iris nhằm nghiên cứu khả năng phân biệt các loài dựa trên đồng thời nhiều đặc trưng thay vì chỉ dựa vào một đặc điểm riêng lẻ.



Hình 1.1: Hình ảnh ba loài hoa được sử dụng trong tập dữ liệu Iris

Bài toán được xây dựng dựa trên ba loài hoa gồm *Iris setosa*, *Iris versicolor* và *Iris virginica*. Mỗi mẫu hoa được mô tả thông qua bốn đặc trưng hình thái: chiều dài đài hoa (Sepal Length), chiều rộng đài hoa (Sepal Width), chiều dài cánh hoa (Petal Length) và chiều rộng

cánh hoa (Petal Width). Bốn giá trị này tạo thành vector đặc trưng đầu vào của mô hình, trong khi tên loài hoa là nhãn cần dự đoán.

Có thể biểu diễn một mẫu dữ liệu Iris dưới dạng:

$$X=(x_1, x_2, x_3, x_4) \quad X=(x_1, x_2, x_3, x_4)$$

trong đó x_1 , x_2 , x_3 và x_4 lần lượt biểu diễn chiều dài đài hoa, chiều rộng đài hoa, chiều dài cánh hoa và chiều rộng cánh hoa.

Nhãn của mẫu thuộc một trong ba lớp:

$$Y=\{\text{Setosa}, \text{Versicolor}, \text{Virginica}\} \quad Y=\{\text{Setosa}, \text{Versicolor}, \text{Virginica}\}$$

Do đó, mục tiêu của mô hình là xây dựng một hàm dự đoán:

$$f: X \rightarrow Y \quad f: X \rightarrow Y$$

Khi một mẫu hoa mới được đưa vào hệ thống, mô hình sẽ nhận bốn giá trị đo được của mẫu đó và xác định mẫu thuộc lớp nào:

$$\hat{y} = f(X) \quad \hat{y} = f(X)$$

Về bản chất, đây là một bài toán **phân loại đa lớp (Multi-class Classification)** vì biến đầu ra có ba khả năng khác nhau. Khác với bài toán phân loại nhị phân chỉ cần quyết định một mẫu thuộc một trong hai nhóm, mô hình trong bài toán Iris phải đồng thời phân biệt ba lớp.

Điểm đáng chú ý của tập dữ liệu Iris là mức độ phân tách giữa các lớp không hoàn toàn giống nhau. *Iris setosa* có các đặc trưng về cánh hoa khác biệt tương đối rõ so với hai loài còn lại, do đó việc nhận diện lớp này thường tương đối dễ. Trong khi đó, *Iris versicolor* và *Iris virginica* có sự tương đồng nhất định về kích thước và đặc điểm hình thái. Khi biểu diễn dữ liệu trong không gian đặc trưng, hai lớp này có thể xuất hiện ở những vùng gần nhau và gây ra một số trường hợp phân loại nhầm.

Hình này có vai trò quan trọng trong việc giải thích bài toán. Có thể nhận thấy các mẫu *Iris setosa* tập trung thành một vùng tương đối riêng biệt, trong khi *Iris versicolor* và *Iris virginica* có sự gần nhau nhiều hơn. Chính cấu trúc này tạo ra những mức độ khó khác nhau cho các thuật toán phân loại.

Từ góc độ học máy, vấn đề không phải là làm cho mô hình ghi nhớ toàn bộ các mẫu đã có mà là tìm ra quy luật đủ tổng quát để có thể xử lý những mẫu chưa từng xuất hiện. Nếu mô hình chỉ hoạt động tốt trên dữ liệu dùng để huấn luyện nhưng cho kết quả kém trên dữ liệu mới, mô hình được xem là có khả năng tổng quát hóa thấp và có thể đã xảy ra hiện tượng quá khớp (Overfitting).

Vì vậy, việc xây dựng bài toán Iris không chỉ dừng lại ở quá trình huấn luyện một thuật toán phân loại. Một quy trình thực nghiệm hoàn chỉnh cần bao gồm việc phân tích dữ liệu, chia dữ liệu thành tập huấn luyện và tập kiểm tra, lựa chọn thuật toán, huấn luyện mô hình, thực hiện dự đoán và đánh giá kết quả. Trong phạm vi đề tài này, phương pháp Hold-out được sử dụng để chia dữ liệu, sau đó bốn thuật toán SVM, Decision Tree, Naive Bayes và K-Nearest Neighbors được áp dụng để thực hiện phân loại.

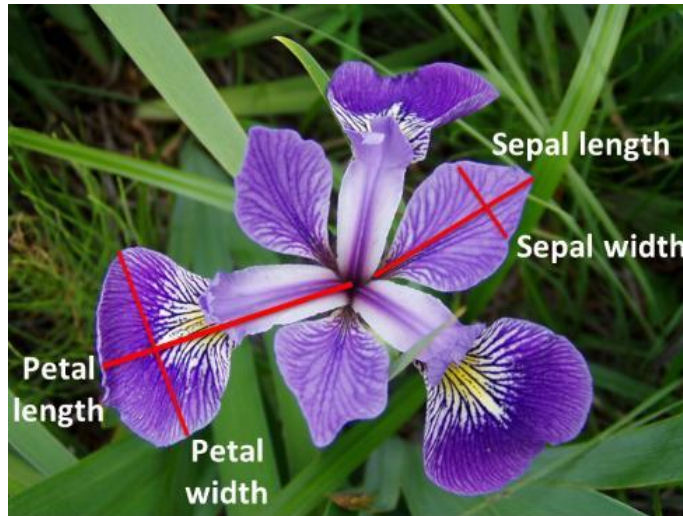
Việc lựa chọn nhiều thuật toán cho cùng một bài toán cho phép quan sát sự khác biệt giữa các phương pháp. SVM tìm kiếm ranh giới phân loại có margin tối ưu; Decision Tree xây dựng các luật phân loại dựa trên các điều kiện trên đặc trưng; Naive Bayes tiếp cận bài toán từ góc độ xác suất; còn KNN xác định lớp của một mẫu dựa trên những mẫu gần nó nhất trong không gian đặc trưng.

Như vậy, bài toán Iris có quy mô nhỏ nhưng bao quát được nhiều khái niệm nền tảng của học máy. Đây là cơ sở để sử dụng tập dữ liệu này làm môi trường thực nghiệm và so sánh các thuật toán phân loại trong những chương tiếp theo.

1.2. Mô tả tập dữ liệu Iris

Tập dữ liệu Iris là một tập dữ liệu dạng bảng, trong đó mỗi dòng biểu diễn một mẫu hoa và mỗi cột biểu diễn một đặc trưng hoặc nhãn tương ứng. Phiên bản dữ liệu được sử dụng phổ biến hiện nay gồm **150 mẫu**, được chia thành ba lớp với số lượng bằng nhau. Mỗi lớp gồm 50 mẫu thuộc một loài Iris.

Bốn đặc trưng đầu vào của mỗi mẫu đều là các đại lượng đo bằng centimet. Hai đặc trưng đầu tiên mô tả phần đài hoa, còn hai đặc trưng còn lại mô tả phần cánh hoa. Cấu trúc này cho phép mô hình khai thác đồng thời kích thước của đài hoa và cánh hoa để xác định loài.



Hình 1.2: Minh họa bốn đặc trưng hình thái của hoa Iris

Hình 1.2 giúp làm rõ ý nghĩa của bốn thuộc tính được sử dụng trong dữ liệu. Trong đó, **Sepal Length** là chiều dài của đài hoa, **Sepal Width** là chiều rộng của đài hoa, **Petal Length** là chiều dài của cánh hoa và **Petal Width** là chiều rộng của cánh hoa.

Có thể mô tả cấu trúc của tập dữ liệu bằng bảng sau:

Thuộc tính	Ý nghĩa	Kiểu dữ liệu
Sepal Length	Chiều dài đài hoa	Số thực
Sepal Width	Chiều rộng đài hoa	Số thực
Petal Length	Chiều dài cánh hoa	Số thực
Petal Width	Chiều rộng cánh hoa	Số thực
Species	Loài hoa	Nhãn phân loại

Bốn thuộc tính đầu tiên được sử dụng làm biến đầu vào XX, còn Species là biến mục tiêu YY. Như vậy, dữ liệu có thể được biểu diễn tổng quát dưới dạng:

$$D=\{(X_i,Y_i)\}_{i=1}^{150}D=\{(X_i,Y_i)\}_{i=1}^{150}$$

với:

$$X_i \in \mathbb{R}^4$$

và:

$$Y_i \in \{\text{Setosa}, \text{Versicolor}, \text{Virginica}\}$$

Một mẫu dữ liệu cụ thể có thể được biểu diễn dưới dạng:

$$X_i = (5.1, 3.5, 1.4, 0.2)$$

Nếu mẫu trên có nhãn Setosa, toàn bộ dữ liệu huấn luyện tương ứng sẽ được xem như một cặp:

$$(X_i, Y_i) = ((5.1, 3.5, 1.4, 0.2), \text{Setosa})$$

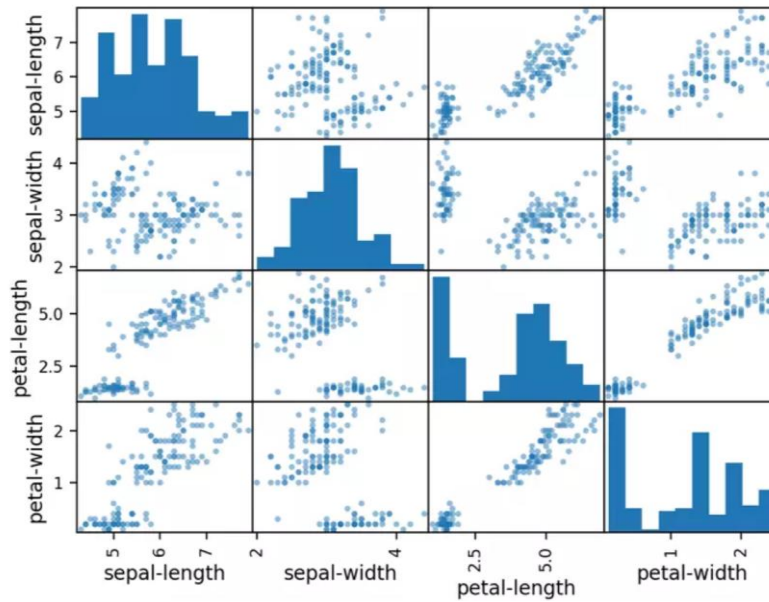
Đối với một bài toán học máy có giám sát, việc tồn tại nhãn của các mẫu trong quá trình huấn luyện là yếu tố quan trọng. Thuật toán không được cung cấp sẵn quy tắc phân loại mà phải tìm ra mối quan hệ giữa bốn đặc trưng và nhãn thông qua các mẫu đã biết.

Một đặc điểm thuận lợi của Iris là dữ liệu tương đối cân bằng. Ba lớp đều có cùng số lượng mẫu nên hiện tượng mất cân bằng lớp không phải là vấn đề chính trong thực nghiệm này. Điều này cho phép sử dụng Accuracy như một chỉ số đánh giá trực quan, đồng thời kết hợp với Precision, Recall và F1-score để có cái nhìn đầy đủ hơn.

Tuy nhiên, số lượng mẫu nhỏ cũng khiến việc đánh giá cần được thực hiện cẩn thận. Khi chỉ có 150 mẫu, một vài mẫu thay đổi giữa tập huấn luyện và tập kiểm tra cũng có thể làm kết quả thay đổi đáng kể. Đây là lý do cần cố định cách chia dữ liệu trong quá trình thực nghiệm và không nên đánh giá thuật toán chỉ dựa trên một con số duy nhất.

Một đặc điểm quan trọng khác của Iris nằm ở sự phân bố không đồng đều về khả năng phân tách giữa các lớp. Khi xét đến các đặc trưng liên quan đến cánh hoa, *Iris setosa* thường tạo thành một vùng khá riêng biệt. Trong khi đó, *Iris versicolor* và *Iris virginica* có những vùng dữ liệu gần nhau hơn.

Để quan sát rõ hơn đặc điểm này, có thể sử dụng biểu đồ phân tán giữa Petal Length và Petal Width.



Hình 1.3: Biểu đồ phân tán của tập dữ liệu Iris theo Petal Length và Petal Width

Từ góc độ biểu diễn dữ liệu, bốn đặc trưng của mỗi mẫu tạo thành một điểm trong không gian bốn chiều. Mỗi lớp tương ứng với một nhóm điểm trong không gian đó. Nhiệm vụ của thuật toán là tìm ra cách phân chia không gian thành các vùng quyết định sao cho mỗi vùng tương ứng với một lớp.

Cách tiếp cận này có sự khác nhau giữa các thuật toán. SVM tìm kiếm một ranh giới phân chia tối ưu, Decision Tree chia không gian thành các vùng dựa trên các ngưỡng của đặc trưng, KNN xác định lớp dựa trên khoảng cách đến những điểm dữ liệu gần nhất, trong khi Naive Bayes tính toán xác suất một mẫu thuộc về từng lớp.

Do đó, mặc dù tập Iris chỉ có bốn đặc trưng, nó vẫn đủ để thể hiện một cách trực quan mối quan hệ giữa dữ liệu đầu vào và bài toán phân loại. Đây cũng là cơ sở để lựa chọn Iris làm tập dữ liệu thực nghiệm trong đề tài.

1.3. Mục tiêu

Mục tiêu chính của đề tài là nghiên cứu khả năng áp dụng các thuật toán học máy có giám sát vào bài toán phân loại hoa Iris, đồng thời so sánh hiệu quả của các thuật toán dựa trên cùng một tập dữ liệu và cùng một quy trình đánh giá.

Trước hết, đề tài hướng đến việc tìm hiểu bản chất của bài toán phân loại Iris và cách dữ liệu được biểu diễn dưới dạng các đặc trưng số. Thông qua bốn thuộc tính về chiều dài và chiều rộng của đài hoa, cánh hoa, mô hình cần học được mối quan hệ giữa đặc trưng đầu vào và ba nhãn loài hoa tương ứng.

Tiếp theo, đề tài sử dụng phương pháp Hold-out để phân chia dữ liệu thành tập huấn luyện và tập kiểm tra. Tập huấn luyện được sử dụng để mô hình tìm hiểu quy luật phân loại, trong khi tập kiểm tra được giữ độc lập nhằm đánh giá khả năng dự đoán đối với những mẫu chưa được sử dụng trong quá trình huấn luyện.

Trên cùng một dữ liệu thực nghiệm, bốn thuật toán SVM, Decision Tree, Naive Bayes và KNN được triển khai. Việc sử dụng nhiều thuật toán nhằm làm rõ sự khác biệt trong cách mỗi phương pháp xây dựng quyết định phân loại. Qua đó, kết quả không chỉ cho biết thuật toán nào đạt hiệu quả cao hơn mà còn giúp giải thích tại sao một phương pháp có thể phù hợp với cấu trúc dữ liệu hơn một phương pháp khác.

Một mục tiêu quan trọng khác là đánh giá mô hình bằng nhiều tiêu chí. Accuracy được sử dụng để xác định tỷ lệ dự đoán đúng trên toàn bộ tập kiểm tra. Precision cho biết mức độ chính xác của các dự đoán thuộc từng lớp, Recall phản ánh khả năng phát hiện các mẫu thực sự thuộc lớp đó, còn F1-score cung cấp một thước đo kết hợp giữa Precision và Recall.

Đối với một lớp cụ thể, các chỉ số có thể được mô tả thông qua:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

và:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Trong bài toán ba lớp, các chỉ số Precision, Recall và F1-score có thể được tổng hợp theo phương pháp **macro average**, trong đó kết quả của từng lớp được tính riêng rồi lấy trung bình. Cách đánh giá này đảm bảo mỗi lớp có mức độ ảnh hưởng tương đương đến kết quả tổng thể.

Bên cạnh việc đánh giá kết quả dự đoán, đề tài còn hướng đến việc nghiên cứu ảnh hưởng của các tham số đến hiệu quả của mô hình. Đối với SVM, các tham số như CC, kernel và γ /gamma có thể ảnh hưởng đến ranh giới quyết định. Đối với Decision Tree, độ sâu của cây có thể quyết định mức độ phức tạp của mô hình. Đối với KNN, số lượng hàng xóm KK ảnh hưởng trực tiếp đến kết quả phân loại. Trong khi đó, Naive Bayes có ít tham số điều chỉnh hơn và chủ yếu dựa trên các giả định xác suất của mô hình.

Cuối cùng, từ kết quả thực nghiệm, đề tài tiến hành so sánh bốn phương pháp và lựa chọn cấu hình phù hợp nhất đối với tập dữ liệu Iris. Việc lựa chọn không chỉ dựa trên Accuracy mà còn xem xét Precision, Recall, F1-score và ma trận nhầm lẫn nhằm xác định mức độ ổn định của mô hình trên từng lớp.

Thông qua quá trình này, đề tài hướng tới việc xây dựng một quy trình thực nghiệm hoàn chỉnh cho bài toán học máy có giám sát. Từ một tập dữ liệu ban đầu, quá trình được triển khai theo trình tự phân tích dữ liệu, chia dữ liệu, xây dựng mô hình, huấn luyện, dự đoán, đánh giá và tối ưu tham số. Đây cũng chính là nền tảng để tiếp tục nghiên cứu sâu hơn về các thuật toán phân loại trong những bài toán có dữ liệu lớn và phức tạp hơn.

CHƯƠNG 2: PHƯƠNG PHÁP CHIA DỮ LIỆU (Hold-out)

2.1. Giới thiệu kỹ thuật Hold-out

Trong quá trình xây dựng một mô hình học máy, việc đánh giá khả năng hoạt động của mô hình trên dữ liệu chưa từng được sử dụng trong quá trình huấn luyện là một yêu cầu quan trọng. Nếu mô hình được huấn luyện và đánh giá trên cùng một tập dữ liệu, kết quả thu được có thể không phản ánh chính xác khả năng dự đoán của mô hình đối với dữ liệu mới. Một mô hình có thể đạt độ chính xác rất cao trên dữ liệu huấn luyện nhưng lại không duy trì được kết quả đó khi được áp dụng cho những mẫu chưa từng quan sát. Hiện tượng này thường liên quan đến vấn đề quá khớp (Overfitting).

Để hạn chế cách đánh giá không chính xác này, dữ liệu ban đầu thường được phân chia thành những tập có mục đích khác nhau. Trong phạm vi đề tài, phương pháp được sử dụng là Hold-out, một kỹ thuật đơn giản và phổ biến để tách dữ liệu thành tập huấn luyện (Training Set) và tập kiểm tra (Test Set).

Về nguyên tắc, tập dữ liệu ban đầu được ký hiệu là:

$$D = \{(x_i, y_i)\}_{i=1}^N$$

Trong đó x_i là vector đặc trưng của mẫu thứ i , y_i là nhãn tương ứng và N là tổng số mẫu. Với tập dữ liệu Iris:

$$N = 150$$

Phương pháp Hold-out thực hiện việc chia tập D thành hai tập con:

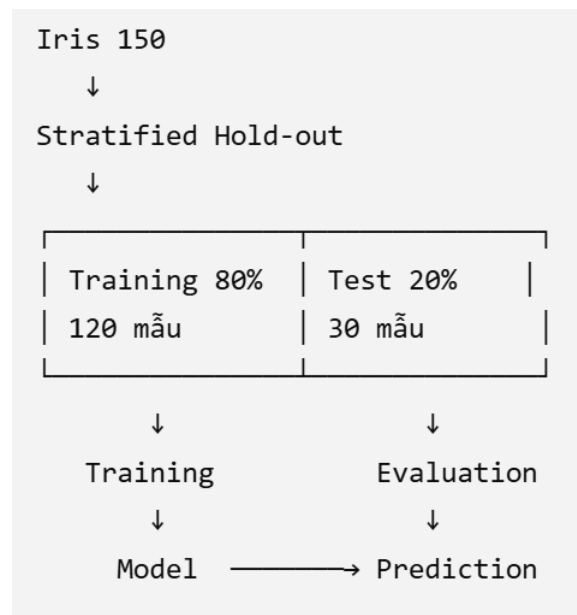
$$D = D_{\text{train}} \cup D_{\text{test}}$$

với điều kiện:

$$D_{\text{train}} \cap D_{\text{test}} = \emptyset$$

Điều này có nghĩa là một mẫu chỉ được xuất hiện trong một trong hai tập. Tập D_{train} được sử dụng để mô hình học mối quan hệ giữa đặc trưng và nhãn, trong khi D_{test} được giữ lại để đánh giá mô hình sau khi quá trình huấn luyện hoàn tất.

Có thể hình dung quá trình này như sau:



Hình 2.4: Quy trình chia dữ liệu bằng phương pháp Hold-out

Hình 2.4 thể hiện rõ vai trò khác nhau của hai tập dữ liệu. Tập huấn luyện được phép tham gia vào quá trình học tham số của mô hình, còn tập kiểm tra không được sử dụng để điều

chính mô hình. Sau khi mô hình hoàn thành quá trình huấn luyện, các mẫu trong tập kiểm tra mới được đưa vào mô hình để tạo ra dự đoán.

Điểm quan trọng của cách tiếp cận này nằm ở tính độc lập tương đối giữa quá trình huấn luyện và quá trình đánh giá. Nếu mô hình chưa từng nhìn thấy các mẫu trong tập kiểm tra, kết quả trên tập này có thể được sử dụng như một cơ sở để ước lượng khả năng mô hình hoạt động trên dữ liệu mới.

Tuy nhiên, Hold-out cũng có một hạn chế là kết quả đánh giá phụ thuộc vào cách dữ liệu được chia. Với một tập dữ liệu nhỏ như Iris, chỉ cần thay đổi một số mẫu giữa tập huấn luyện và tập kiểm tra thì kết quả Accuracy hoặc F1-score cũng có thể thay đổi. Vì vậy, cách chia dữ liệu cần được thiết lập một cách nhất quán trong toàn bộ quá trình thực nghiệm.

2.2. Phương pháp chia dữ liệu Hold-out

Trong đề tài này, tập dữ liệu Iris gồm 150 mẫu được chia thành tập huấn luyện và tập kiểm tra theo tỷ lệ **80:20**. Theo đó, 80% dữ liệu được sử dụng để huấn luyện mô hình và 20% dữ liệu được giữ lại để kiểm tra.

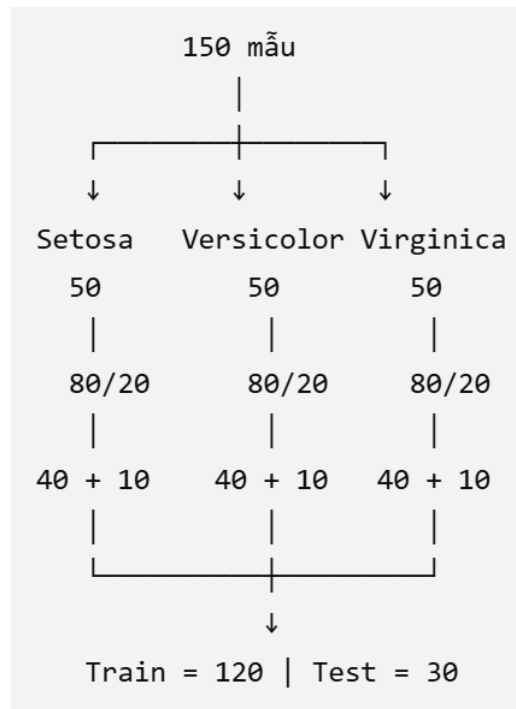
Với tổng số 150 mẫu, số lượng mẫu ở mỗi tập được xác định như sau:

$$N_{\text{train}} = 150 \times 0.8 = 120 \quad N_{\text{train}} = 150 \times 0.8 = 120$$

và:

$$N_{\text{test}} = 150 \times 0.2 = 30 \quad N_{\text{test}} = 150 \times 0.2 = 30$$

Như vậy, tập huấn luyện gồm 120 mẫu và tập kiểm tra gồm 30 mẫu.



Hình 2.5: Tỷ lệ phân chia tập dữ liệu Iris thành tập huấn luyện và tập kiểm tra

Phân chia có phân tầng

Do bài toán Iris là bài toán phân loại đa lớp, việc chia dữ liệu cần đảm bảo rằng ba lớp *Iris setosa*, *Iris versicolor* và *Iris virginica* vẫn được đại diện trong cả tập huấn luyện và tập kiểm tra.

Đây là lý do phương pháp **stratified splitting** được sử dụng. Thay vì chỉ lấy ngẫu nhiên một số lượng mẫu nhất định, phương pháp phân tầng cố gắng duy trì tỷ lệ của các lớp trong tập dữ liệu ban đầu khi tạo ra hai tập con.

Ban đầu, mỗi lớp Iris có 50 mẫu. Khi chia theo tỷ lệ 80:20, có thể thu được khoảng 40 mẫu của mỗi lớp trong tập huấn luyện và 10 mẫu của mỗi lớp trong tập kiểm tra. Khi đó:

$$D_{\text{train}}=120 \text{ mẫu}$$

gồm khoảng:

$$40 \text{ Setosa} + 40 \text{ Versicolor} + 40 \text{ Virginica}$$

và:

$$D_{\text{test}}=30 \text{ mẫu}$$

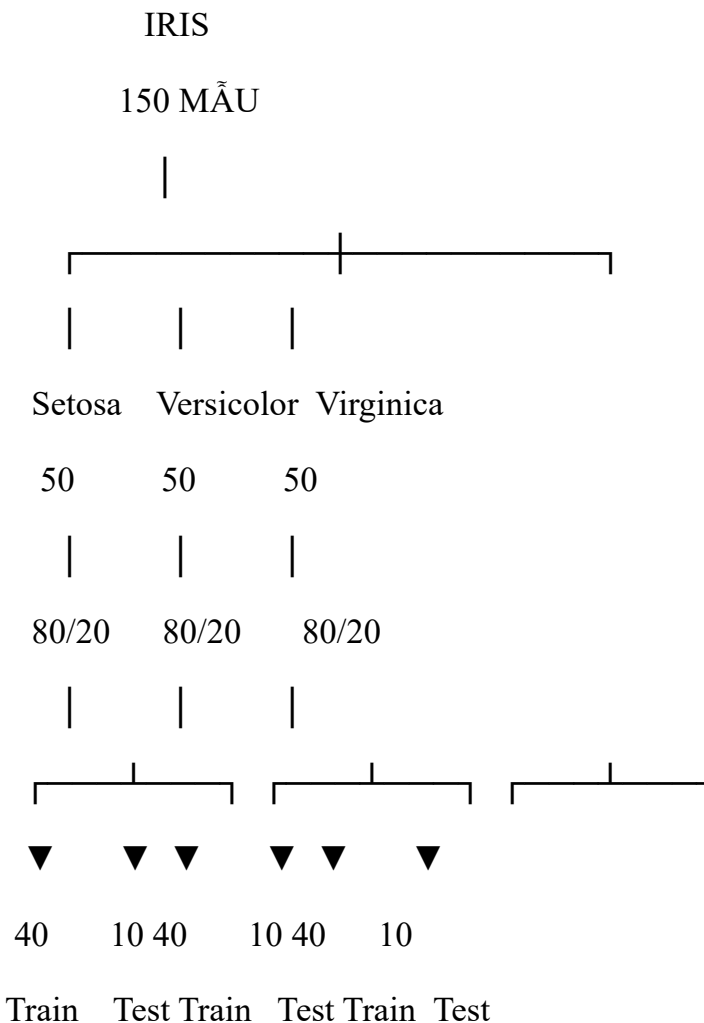
gồm khoảng:

10 Setosa+10 Versicolor+10 Virginica 10\ Setosa+10\ Versicolor+10\ Virginica

Cách chia này có ý nghĩa quan trọng đối với việc đánh giá mô hình. Nếu việc lấy mẫu hoàn toàn ngẫu nhiên khiến một lớp có quá ít mẫu trong tập kiểm tra, kết quả đánh giá lớp đó sẽ kém ổn định và không phản ánh đúng khả năng phân loại. Với Stratified Hold-out, mỗi lớp đều được đại diện trong tập kiểm tra, từ đó kết quả Precision, Recall và F1-score của từng lớp có ý nghĩa hơn.

[CHÈN HÌNH 2.2 – NẾU MUỐN CÓ]

Bạn có thể tự tạo một hình như:



Hình này **rất phù hợp với luận văn** vì nó thể hiện được bản chất của Stratified Hold-out chứ không chỉ trang trí.

Vai trò của Random State

Một vấn đề khác trong quá trình chia dữ liệu là tính ngẫu nhiên. Khi thực hiện Hold-out, các mẫu được lựa chọn một cách ngẫu nhiên. Nếu không cố định trạng thái ngẫu nhiên, mỗi lần chạy chương trình có thể tạo ra một cách chia dữ liệu khác nhau, dẫn đến kết quả của mô hình cũng có thể thay đổi.

Để đảm bảo khả năng tái lập kết quả, đề tài sử dụng một giá trị `random_state` cố định. Khi cùng một tập dữ liệu, cùng tỷ lệ chia và cùng giá trị `random_state` được sử dụng, quá trình chia dữ liệu có thể được tái tạo trong những lần thực nghiệm khác nhau.

Ví dụ, khi sử dụng thư viện Scikit-learn, quá trình chia dữ liệu có thể được thực hiện theo nguyên tắc:

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
)
```

Trong đoạn mã trên, `test_size=0.2` quy định 20% dữ liệu được sử dụng làm tập kiểm tra. `random_state=42` giúp cố định cách chia dữ liệu, còn `stratify=y` đảm bảo tỷ lệ giữa các lớp được duy trì khi phân chia.

Trong một báo cáo học thuật, giá trị 42 không có ý nghĩa đặc biệt về mặt toán học. Đây đơn giản là một giá trị được lựa chọn để quá trình thực nghiệm có thể tái lập. Điều quan trọng là phải sử dụng **cùng một giá trị trong toàn bộ quá trình so sánh các mô hình**.

Tại sao không huấn luyện trên toàn bộ 150 mẫu?

Một câu hỏi có thể đặt ra là nếu có 150 mẫu thì tại sao không sử dụng toàn bộ 150 mẫu để huấn luyện nhằm cung cấp cho mô hình nhiều dữ liệu nhất có thể.

Vấn đề nằm ở việc khi toàn bộ dữ liệu được sử dụng để huấn luyện, sẽ không còn một tập dữ liệu độc lập để đánh giá khả năng tổng quát hóa. Khi đó, nếu mô hình dự đoán đúng 145 trên 150 mẫu, con số này chỉ cho biết mô hình hoạt động tốt trên những dữ liệu mà nó đã được nhìn thấy, chứ chưa đủ cơ sở để kết luận rằng mô hình sẽ hoạt động tốt trên những mẫu mới.

Hold-out giải quyết vấn đề này bằng cách chủ động giữ lại một phần dữ liệu. Ba mươi mẫu kiểm tra trong nghiên cứu không tham gia vào quá trình học của mô hình. Sau khi mô hình được huấn luyện, các mẫu này mới được đưa vào để đánh giá.

Có thể mô tả toàn bộ quy trình:

$$D \rightarrow D_{\text{train}} + D_{\text{test}} \quad D \rightarrow D_{\text{train}} + D_{\text{test}}$$

sau đó:

$$D_{\text{train}} \rightarrow \text{Training Model } D_{\text{train}} \rightarrow \text{Model}$$

và cuối cùng:

$$\text{Model} + D_{\text{test}} \rightarrow \text{Prediction } \hat{Y} \quad \text{Model} + D_{\text{test}} \rightarrow \text{Prediction } \hat{Y}$$

Các giá trị dự đoán $\hat{Y}$ được so sánh với nhãn thực tế Y để tính các chỉ số đánh giá.

Ý nghĩa của Hold-out đối với thực nghiệm

- Trong phạm vi đề tài, Hold-out đóng vai trò là cầu nối giữa quá trình xây dựng mô hình và quá trình đánh giá mô hình. Việc tất cả các thuật toán SVM, Decision Tree, Naive Bayes và KNN được đánh giá trên cùng một tập kiểm tra giúp quá trình so sánh trở nên nhất quán hơn.
- Nếu mỗi thuật toán được sử dụng một cách chia dữ liệu khác nhau, sự khác biệt về kết quả có thể không hoàn toàn đến từ thuật toán mà còn đến từ sự khác biệt trong dữ liệu huấn luyện và kiểm tra. Ngược lại, khi bốn thuật toán sử dụng cùng một tập huấn luyện và cùng một tập kiểm tra, kết quả có cơ sở tốt hơn để phản ánh sự khác biệt giữa các phương pháp.

- Tuy nhiên, cần lưu ý rằng Hold-out vẫn có sự phụ thuộc vào một lần chia dữ liệu duy nhất. Đây là hạn chế đặc biệt đáng chú ý đối với những tập dữ liệu nhỏ như Iris. Trong các nghiên cứu cần đánh giá ổn định hơn, có thể sử dụng các phương pháp như **k-fold Cross-Validation**, trong đó dữ liệu được chia thành nhiều phần và mô hình được huấn luyện, đánh giá nhiều lần trên các cách phân chia khác nhau.
- Trong phạm vi đề tài, Hold-out được lựa chọn bởi phương pháp này có cách triển khai đơn giản, dễ giải thích và phù hợp với mục tiêu minh họa quy trình phân loại cơ bản. Việc sử dụng Stratified Hold-out đồng thời giúp hạn chế ảnh hưởng của sự mất cân bằng giữa các lớp trong quá trình đánh gi

CHƯƠNG 3: GIỚI THIỆU MÔ HÌNH VÀ THUẬT TOÁN

Trong bài toán phân loại tập dữ liệu Iris, việc lựa chọn thuật toán học máy có ảnh hưởng trực tiếp đến khả năng dự đoán cũng như chất lượng của mô hình. Mặc dù tập dữ liệu Iris có kích thước tương đối nhỏ và cấu trúc dữ liệu khá đơn giản, đây vẫn là một tập dữ liệu phù hợp để nghiên cứu và so sánh nhiều phương pháp phân loại khác nhau. Mỗi thuật toán có một cách tiếp cận riêng đối với mối quan hệ giữa các đặc trưng đầu vào và nhãn lớp đầu ra.

Trong nghiên cứu này, bốn thuật toán phân loại được lựa chọn để thực nghiệm gồm Support Vector Machine (SVM), Decision Tree, Naive Bayes và K-Nearest Neighbors (KNN). Việc lựa chọn bốn mô hình này nhằm tạo ra sự đa dạng về phương pháp tiếp cận. SVM tìm kiếm một ranh giới phân chia tối ưu giữa các lớp, Decision Tree xây dựng mô hình dưới dạng cấu trúc cây dựa trên các điều kiện phân chia dữ liệu, Naive Bayes sử dụng mô hình xác suất dựa trên định lý Bayes, trong khi KNN thực hiện phân loại dựa trên mức độ gần nhau giữa các mẫu dữ liệu.

Thông qua việc áp dụng các mô hình trên cùng một tập dữ liệu và cùng một phương pháp đánh giá, có thể quan sát được sự khác biệt về khả năng phân loại, mức độ phù hợp của từng thuật toán cũng như ảnh hưởng của các tham số đến kết quả dự đoán.

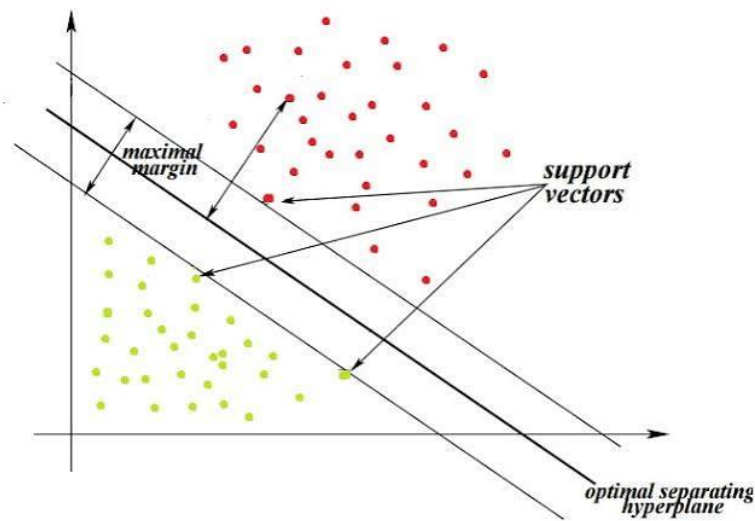
3.1. SVM (Support Vector Machine)

Support Vector Machine, viết tắt là SVM, là một trong những thuật toán học máy có giám sát phổ biến được sử dụng cho các bài toán phân loại và hồi quy. Ý tưởng cơ bản của SVM là tìm một siêu phẳng có khả năng phân chia các lớp dữ liệu sao cho khoảng cách từ siêu phẳng đến các điểm dữ liệu gần nhất của mỗi lớp là lớn nhất.

Đối với bài toán phân loại hai lớp, giả sử dữ liệu được biểu diễn bởi các điểm trong không gian đặc trưng. Nếu tồn tại một đường thẳng hoặc một siêu phẳng có thể phân chia hoàn toàn hai lớp, SVM không chỉ tìm một đường phân chia bất kỳ mà cố gắng tìm đường phân chia có biên lớn nhất. Biên, hay Margin, là khoảng cách giữa siêu phẳng phân loại và những điểm dữ liệu gần siêu phẳng nhất.

Các điểm dữ liệu nằm gần biên và có vai trò quyết định vị trí của siêu phẳng được gọi là Support Vector. Đây là những điểm quan trọng nhất đối với mô hình SVM. Nếu các điểm dữ

liệu khác thay đổi vị trí nhưng các Support Vector không thay đổi đáng kể, ranh giới phân loại của mô hình thường cũng không thay đổi nhiều.



Hình 3.6: Nguyên lý phân loại của SVM với siêu phẳng, Margin và các Support Vector

Về mặt toán học, với một bài toán phân loại tuyến tính, siêu phẳng có thể được biểu diễn dưới dạng:

$$\mathbf{w}^T \mathbf{x} + b = 0$$

Trong đó ($\mathbf{w}$) là vector trọng số, ($\mathbf{x}$) là vector đặc trưng của một mẫu dữ liệu và (b) là hệ số điều chỉnh vị trí của siêu phẳng. SVM tìm ($\mathbf{w}$) và (b) sao cho các mẫu thuộc hai lớp nằm về hai phía khác nhau của siêu phẳng đồng thời tối đa hóa khoảng cách giữa các lớp.

Trong thực tế, dữ liệu không phải lúc nào cũng có thể phân chia hoàn toàn bằng một đường thẳng hoặc một siêu phẳng. Khi đó, SVM sử dụng Soft Margin, cho phép một số điểm dữ liệu nằm sai phía của biên nhằm tạo ra mô hình có khả năng tổng quát tốt hơn. Tham số (C) được sử dụng để điều chỉnh sự đánh đổi giữa việc tối đa hóa Margin và việc hạn chế các mẫu bị phân loại sai.

Khi (C) có giá trị lớn, mô hình có xu hướng cố gắng phân loại chính xác các mẫu trong tập huấn luyện, do đó biên có thể trở nên hẹp hơn. Ngược lại, khi (C) nhỏ, mô hình chấp nhận một số lỗi trên tập huấn luyện để có được biên rộng hơn và có khả năng tổng quát tốt hơn.

Một ưu điểm quan trọng của SVM là khả năng xử lý dữ liệu không phân tách tuyến tính thông qua Kernel. Kernel cho phép ánh xạ dữ liệu từ không gian ban đầu sang một không gian đặc trưng có số chiều cao hơn mà không nhất thiết phải thực hiện trực tiếp phép biến đổi đó.

Một Kernel phổ biến là Radial Basis Function, viết tắt là RBF. Hàm Kernel RBF được biểu diễn bởi:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

Trong đó (γ) quyết định mức độ ảnh hưởng của một điểm dữ liệu đối với các điểm xung quanh. Giá trị (γ) lớn khiến vùng ảnh hưởng của từng mẫu nhỏ hơn, trong khi giá trị (γ) nhỏ tạo ra vùng ảnh hưởng rộng hơn.

Đối với tập dữ liệu Iris, SVM đặc biệt phù hợp vì dữ liệu chỉ có bốn đặc trưng số và một số lớp có sự phân tách khá rõ ràng trong không gian đặc trưng. Tuy nhiên, do SVM dựa nhiều vào khoảng cách và hình học của dữ liệu, việc chuẩn hóa các đặc trưng trước khi huấn luyện thường là một bước quan trọng, đặc biệt khi các đặc trưng có thang đo hoặc độ phân tán khác nhau.

Trong bài toán phân loại nhiều lớp như Iris, SVM có thể mở rộng từ bài toán hai lớp sang nhiều lớp bằng các chiến lược phân loại đa lớp. Trong thư viện Scikit-learn, mô hình SVC sử dụng chiến lược One-vs-One cho bài toán nhiều lớp.

Nhìn chung, SVM có ưu điểm là khả năng tạo ra ranh giới phân loại mạnh, hoạt động tốt với dữ liệu có số chiều không quá lớn và có thể xử lý các quan hệ phi tuyến thông qua Kernel. Tuy nhiên, việc lựa chọn các tham số như (C) và (γ) có ảnh hưởng đáng kể đến kết quả. Nếu lựa chọn không phù hợp, mô hình có thể bị Underfitting hoặc Overfitting.

3.2. Decision Tree

Decision Tree, hay cây quyết định, là một thuật toán học máy có giám sát sử dụng cấu trúc dạng cây để thực hiện phân loại hoặc dự đoán. Khác với SVM, Decision Tree không tìm một siêu phẳng phân chia dữ liệu mà liên tục đặt ra các điều kiện để chia tập dữ liệu thành những nhóm nhỏ hơn.

Một cây quyết định thông thường bao gồm nút gốc, các nút bên trong, các nhánh và các nút lá. Nút gốc là điểm bắt đầu của quá trình phân loại. Mỗi nút bên trong tương ứng với một điều kiện phân chia dữ liệu dựa trên một đặc trưng. Các nhánh thể hiện kết quả của điều kiện đó, còn nút lá biểu diễn kết quả dự đoán cuối cùng.

Ví dụ, đối với tập dữ liệu Iris, cây quyết định có thể phát hiện rằng chiều dài hoặc chiều rộng cánh hoa có khả năng phân biệt các loài hoa khá tốt. Mô hình có thể xây dựng một quy tắc dạng:

Nếu Petal Length nhỏ hơn một ngưỡng nhất định thì mẫu có khả năng thuộc lớp Setosa. Nếu không, mô hình tiếp tục kiểm tra một đặc trưng khác để phân biệt Versicolor và Virginica.

Để lựa chọn cách chia dữ liệu tại mỗi nút, Decision Tree sử dụng các tiêu chí đánh giá mức độ hỗn tạp của dữ liệu. Hai tiêu chí phổ biến là Gini Impurity và Entropy.

Chỉ số Gini được tính theo công thức:

$$\text{Gini} = 1 - \sum_{i=1}^K p_i^2$$

Trong đó (K) là số lớp và (p_i) là tỷ lệ mẫu thuộc lớp (i) trong nút đang xét. Nếu một nút chỉ chứa các mẫu thuộc cùng một lớp, chỉ số Gini bằng 0, nghĩa là nút đó hoàn toàn thuần nhất. Giá trị Gini càng nhỏ thì mức độ hỗn tạp của nút càng thấp.

Entropy được xác định bởi:

$$\text{Entropy} = - \sum_{i=1}^K p_i \log_2(p_i)$$

Entropy phản ánh mức độ không chắc chắn của một nút. Khi tất cả các mẫu thuộc cùng một lớp, Entropy bằng 0. Khi các lớp được phân bố đồng đều, Entropy đạt giá trị lớn hơn.

Dựa trên các tiêu chí này, thuật toán tìm cách lựa chọn đặc trưng và ngưỡng chia sao cho dữ liệu sau khi phân chia trở nên thuần nhất hơn. Quá trình này được thực hiện lặp lại trên các nút con để hình thành cấu trúc cây.

Một vấn đề quan trọng của Decision Tree là Overfitting. Nếu cây được phép phát triển quá sâu, mô hình có thể học quá chi tiết các đặc điểm của tập huấn luyện, bao gồm cả những biến động hoặc nhiễu không mang tính tổng quát. Khi đó, độ chính xác trên tập huấn luyện có thể rất cao nhưng hiệu quả trên dữ liệu chưa từng thấy lại giảm.

Để kiểm soát vấn đề này, một số tham số thường được sử dụng như (`max_depth`), (`min_samples_split`) và (`min_samples_leaf`). Trong đó, (`max_depth`) giới hạn độ sâu tối đa của cây, còn (`min_samples_leaf`) quy định số lượng mẫu tối thiểu cần có tại một nút lá.

Ưu điểm lớn của Decision Tree là mô hình dễ hiểu và dễ trực quan hóa. Các quy tắc phân loại có thể được biểu diễn dưới dạng các điều kiện rõ ràng, giúp người sử dụng dễ dàng giải thích kết quả. Ngoài ra, Decision Tree không yêu cầu dữ liệu phải được chuẩn hóa như các thuật toán dựa trên khoảng cách.

Tuy nhiên, Decision Tree có thể thay đổi đáng kể nếu dữ liệu huấn luyện thay đổi. Cây quá sâu cũng dễ bị Overfitting. Vì vậy, việc lựa chọn độ sâu và các tham số kiểm soát kích thước cây là một phần quan trọng trong quá trình xây dựng mô hình.

3.3. Naive Bayes

Naive Bayes là nhóm thuật toán phân loại dựa trên định lý Bayes trong xác suất. Khác với SVM và Decision Tree, Naive Bayes tiếp cận bài toán phân loại theo hướng xác suất. Với một mẫu dữ liệu mới, mô hình tính xác suất mẫu đó thuộc về từng lớp và lựa chọn lớp có xác suất hậu nghiệm lớn nhất.

Định lý Bayes được biểu diễn như sau:

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

Trong đó (C) là một lớp cần dự đoán và (X) là vector các đặc trưng của mẫu dữ liệu. ($P(C|X)$) là xác suất hậu nghiệm của lớp (C) khi đã biết các đặc trưng (X). ($P(X|C)$) là xác suất quan sát được các đặc trưng (X) khi mẫu thuộc lớp (C). ($P(C)$) là xác suất tiên nghiệm của lớp và ($P(X)$) là xác suất xuất hiện của mẫu dữ liệu.

Điểm đặc biệt của Naive Bayes nằm ở giả định rằng các đặc trưng độc lập có điều kiện với nhau khi đã biết lớp. Với vector đặc trưng của Iris gồm chiều dài đài hoa, chiều rộng đài hoa, chiều dài cánh hoa và chiều rộng cánh hoa, mô hình giả định rằng các đặc trưng này có thể được xem xét độc lập có điều kiện theo từng lớp.

Mặc dù giả định độc lập này thường không hoàn toàn đúng trong dữ liệu thực tế, Naive Bayes vẫn có thể đạt kết quả tốt trong nhiều bài toán. Đây cũng chính là lý do thuật toán được gọi là “Naive”, tức là sử dụng một giả định đơn giản hóa về mối quan hệ giữa các đặc trưng.

Đối với dữ liệu Iris, các đặc trưng đều là dữ liệu số liên tục nên một lựa chọn phù hợp là Gaussian Naive Bayes. Mô hình giả định rằng giá trị của mỗi đặc trưng trong từng lớp tuân theo phân phối Gaussian.

Hàm mật độ xác suất Gaussian được biểu diễn:

$$P(x_j|C) = \frac{1}{\sqrt{2\pi}\sigma_{C,j}^2}} \exp\left(-\frac{(x_j - \mu_{C,j})^2}{2\sigma_{C,j}^2}\right)$$

Trong đó (x_j) là giá trị của đặc trưng thứ (j) , $(\mu_{C,j})$ là giá trị trung bình của đặc trưng đó đối với lớp (C) , còn $(\sigma_{C,j}^2)$ là phương sai tương ứng.

Quá trình huấn luyện Gaussian Naive Bayes tương đối đơn giản. Mô hình cần ước lượng các tham số xác suất của từng lớp, bao gồm xác suất tiên nghiệm, giá trị trung bình và phương sai của từng đặc trưng trong mỗi lớp. Khi xuất hiện một mẫu mới, mô hình sử dụng các tham số này để tính xác suất mẫu thuộc từng lớp.

Một ưu điểm của Naive Bayes là tốc độ huấn luyện và dự đoán nhanh, đồng thời không yêu cầu mô hình có cấu trúc phức tạp. Thuật toán cũng hoạt động tốt với các tập dữ liệu có số lượng mẫu tương đối nhỏ. Đây là đặc điểm phù hợp với tập dữ liệu Iris.

Tuy nhiên, giả định độc lập giữa các đặc trưng là một hạn chế của phương pháp. Nếu các đặc trưng có mối quan hệ phụ thuộc mạnh, giả định này có thể làm giảm khả năng mô hình hóa chính xác cấu trúc thực tế của dữ liệu. Ngoài ra, đối với Gaussian Naive Bayes, giả định về phân phối Gaussian cũng có thể không phù hợp hoàn toàn với mọi tập dữ liệu.

Trong Scikit-learn, tham số (var_smoothing) được sử dụng để ổn định việc tính toán phương sai và hạn chế các vấn đề số học khi phương sai của một đặc trưng có giá trị quá nhỏ.

3.4. K-Nearest Neighbors (KNN)

K-Nearest Neighbors, viết tắt là KNN, là một thuật toán phân loại dựa trên nguyên lý những mẫu dữ liệu gần nhau trong không gian đặc trưng thường có xu hướng thuộc cùng một lớp. Không giống nhiều thuật toán khác, KNN gần như không xây dựng một mô hình tham số rõ ràng trong quá trình huấn luyện. Thay vào đó, thuật toán lưu lại dữ liệu huấn luyện và thực hiện tính toán khi cần dự đoán một mẫu mới.

Khi xuất hiện một mẫu chưa biết nhãn, KNN tính khoảng cách giữa mẫu đó với các mẫu trong tập huấn luyện. Sau đó, thuật toán lựa chọn (K) mẫu có khoảng cách gần nhất. Nhãn của mẫu mới được xác định dựa trên các nhãn của những láng giềng gần nhất này.

Khoảng cách Euclidean, một trong những khoảng cách phổ biến nhất đối với dữ liệu số, được tính theo công thức:

$$d(x,y)=\sqrt{\sum_{i=1}^n(x_i-y_i)^2}$$

Trong đó (x) và (y) là hai mẫu dữ liệu, còn (n) là số lượng đặc trưng.

Tham số quan trọng nhất của KNN là (K). Nếu (K) có giá trị nhỏ, mô hình tập trung mạnh vào các điểm dữ liệu gần nhất. Điều này giúp mô hình phản ứng tốt với các cấu trúc cục bộ của dữ liệu nhưng đồng thời dễ bị ảnh hưởng bởi nhiễu. Ngược lại, nếu (K) quá lớn, quá nhiều điểm dữ liệu được sử dụng để đưa ra quyết định, khiến ranh giới giữa các lớp trở nên quá đơn giản và có thể dẫn đến Underfitting.

Do đó, việc lựa chọn giá trị (K) phù hợp có ảnh hưởng trực tiếp đến hiệu quả của KNN. Trong quá trình thực nghiệm, nhiều giá trị (K) khác nhau có thể được kiểm tra và lựa chọn dựa trên kết quả đánh giá.

KNN cũng có thể sử dụng các cách gán trọng số khác nhau cho các láng giềng. Với cách gán trọng số đồng đều, mọi láng giềng có mức độ ảnh hưởng như nhau. Với cách gán trọng số theo khoảng cách, những điểm càng gần mẫu cần dự đoán sẽ có ảnh hưởng càng lớn.

Một vấn đề đặc biệt quan trọng đối với KNN là sự khác biệt về thang đo của các đặc trưng. Vì thuật toán dựa trực tiếp trên khoảng cách, một đặc trưng có giá trị lớn hơn về mặt đơn vị hoặc phạm vi có thể chi phối kết quả tính khoảng cách. Do đó, trong thực nghiệm, dữ liệu thường được chuẩn hóa trước khi đưa vào KNN.

Đối với tập dữ liệu Iris, việc sử dụng KNN có ý nghĩa vì số lượng đặc trưng không lớn và các mẫu của cùng một loài thường có xu hướng nằm gần nhau trong không gian đặc trưng. Tuy nhiên, KNN có nhược điểm là chi phí tính toán dự đoán tăng khi kích thước tập dữ liệu tăng, bởi mô hình phải tìm kiếm các láng giềng gần nhất trong tập dữ liệu huấn luyện.

CHƯƠNG 4: THỰC NGHIỆM VÀ KẾT QUẢ

Chương này trình bày toàn bộ quy trình thực nghiệm, kết quả đánh giá chi tiết và các phân tích so sánh giữa bốn mô hình học máy: Support Vector Machine (SVM), Decision Tree, Naive Bayes và K-Nearest Neighbors trên tập dữ liệu hoa Iris. Để đảm bảo tính khách quan và chuẩn mực khoa học, tất cả các mô hình đều được huấn luyện và kiểm thử trên cùng một cấu hình phân chia dữ liệu Hold-out 80:20 Stratified với random_state=42 theo đúng thiết kế đã được xây dựng ở Chương 2.

4.1. Môi trường thực nghiệm

Quá trình thực nghiệm và kiểm thử các mô hình được triển khai trên hệ thống máy tính với cấu hình phần cứng và môi trường phần mềm đồng nhất, được đặc tả chi tiết trong bảng dưới

Thành phần	Thông số kỹ thuật / Phiên bản	Ghi chú / Vai trò
Hệ điều hành	Microsoft Windows 10 Pro (64-bit)	Môi trường thực thi chính
Bộ xử lý (CPU)	Intel / AMD Multi- core High- performance Processor	Xử lý huấn luyện và tính toán ma trận
Bộ nhớ RAM	16 GB High-speed DDR4/DDR5	Lưu trữ dữ liệu và mô hình trong bộ nhớ
Ngôn ngữ lập trình	Python phiên bản 3.14.7 (64-bit)	Ngôn ngữ cốt lõi cho toàn bộ mã nguồn

Thư viện Học máy	Scikit-learn phiên bản 1.9.1	Cung cấp các thuật toán SVM, DT, NB, KNN, GridSearchCV
Thư viện Dữ liệu	Pandas 3.0.5 & NumPy 2.5.3	Xử lý cấu trúc dữ liệu bảng và tính toán ma trận
Thư viện Trực quan	Matplotlib 3.11.2 & Seaborn 0.13.2	Vẽ biểu đồ so sánh, ma trận nhầm lẫn, ranh giới phân loại

4.2. Kết quả từng mô hình (Accuracy, Precision, Recall, F1)

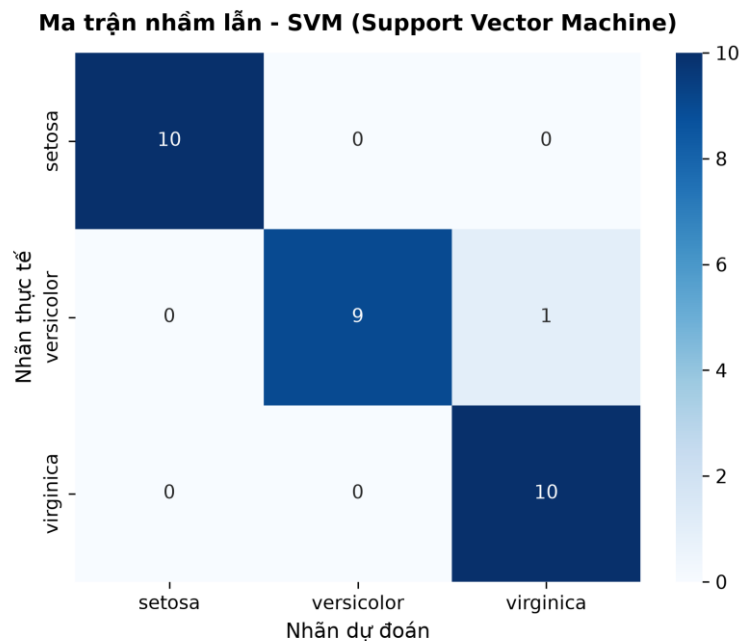
Tập dữ liệu kiểm tra bao gồm 30 mẫu độc lập (10 mẫu Setosa, 10 mẫu Versicolor và 10 mẫu Virginica). Dưới đây là phân tích chi tiết kết quả thực nghiệm đạt được trên từng mô hình học máy cụ thể.

4.2.1. Mô hình *Support Vector Machine (SVM)*

Mô hình SVM sử dụng hàm hạt nhân RBF (Radial Basis Function) với tham số mặc định $C=1.0$ và $\gamma='scale'$. Trên tập kiểm tra gồm 30 mẫu, mô hình đạt độ chính xác tổng thể Accuracy = 0.9667 (tương đương 96.67%). Chỉ số Precision trung bình Macro đạt 0.9697, Recall trung bình đạt 0.9667 và F1-Score trung bình đạt 0.9666. Thời gian huấn luyện mô hình rất nhanh, chỉ mất khoảng 2.71 mili-giây.

Lớp mục tiêu	Precision	Recall	F1-Score	Số mẫu hỗ trợ (Support)
Iris setosa	1.00	1.00	1.00	10

Iris versicolor	1.00	0.90	0.95	10
Iris virginica	0.91	1.00	0.95	10
Macro Average	0.97	0.97	0.97	30



Hình 4.7: Ma trận nhầm lẫn (Confusion Matrix) của mô hình SVM trên tập kiểm tra

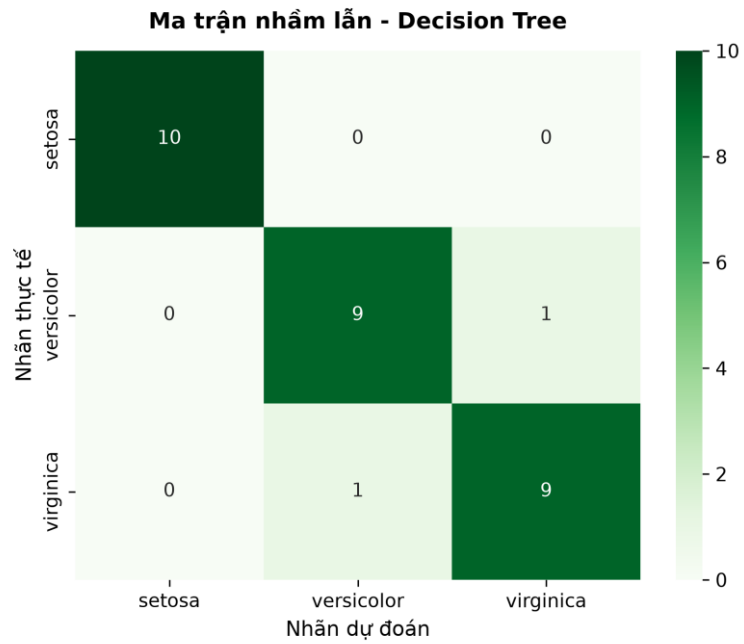
Nhận xét kết quả SVM: Quan sát ma trận nhầm lẫn ở Hình 4.1 cho thấy mô hình phân loại chính xác 100% loài Iris setosa (10/10 mẫu) và Iris virginica (10/10 mẫu). Riêng đối với Iris versicolor, có 9 mẫu được dự đoán đúng và chỉ 1 mẫu bị nhầm lẫn sang Virginica. Điều này phản ánh chính xác bản chất dữ liệu: các đặc trưng hình thái giữa Versicolor và Virginica có một vùng giao thoa nhỏ trong không gian phân bố, trong khi Setosa nằm hoàn toàn biệt lập.

4.2.2. Mô hình Cây quyết định (Decision Tree)

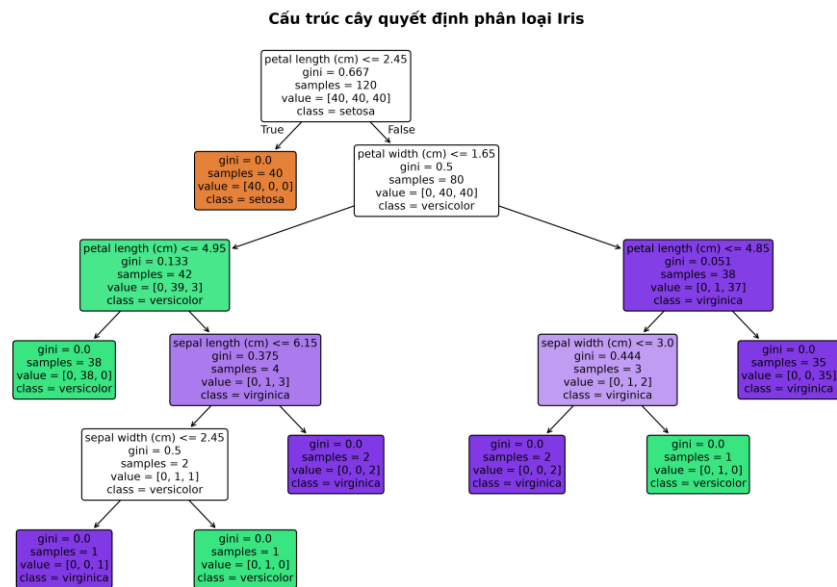
Mô hình Cây quyết định được khởi tạo với tiêu chí phân nhánh Gini Impurity. Kết quả thực nghiệm trên tập kiểm tra cho thấy mô hình đạt độ chính xác Accuracy = 0.9333 (tương đương 93.33%), Precision trung bình Macro = 0.9333, Recall = 0.9333, và F1-Score = 0.9333. Thời gian huấn luyện là 2.23 ms.

Lớp mục tiêu	Precision	Recall	F1-Score	Số mẫu hỗ trợ (Support)
Iris setosa	1.00	1.00	1.00	10
Iris versicolor	0.90	0.90	0.90	10
Iris virginica	0.90	0.90	0.90	10
Macro Average	0.93	0.93	0.93	30

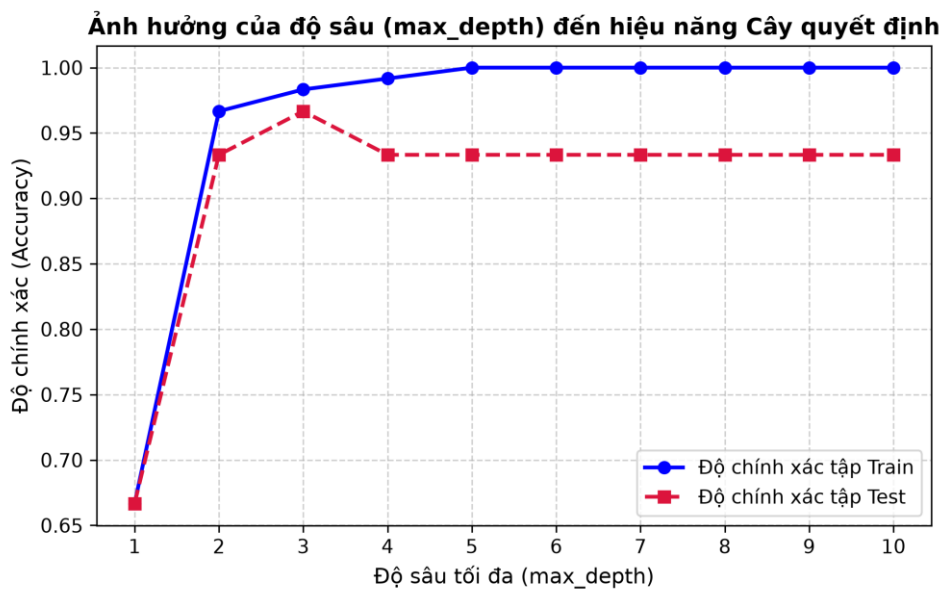
Báo cáo phân loại chi tiết của mô hình Decision Tree



Ma trận nhầm lẫn của mô hình Cây quyết định (Decision Tree)



Sơ đồ cấu trúc cây quyết định phân loại hoa Iris được huấn luyện



Đồ thị biến thiên độ chính xác trên tập Train và Test theo độ sâu tối đa (max_depth)

Nhận xét kết quả Cây quyết định: Nhìn vào cấu trúc cây, ta thấy thuộc tính Petal Width (hoặc Petal Length) được chọn ngay ở nút gốc với ngưỡng 0.8 cm để tách biệt hoàn toàn loài Iris setosa với độ thuần khiết tuyệt đối (Gini = 0.0). Tuy nhiên, giữa Versicolor và Virginica, cây phải phân nhánh thêm nhiều tầng và xảy ra 2 trường hợp nhầm lẫn chéo (1 mẫu Versicolor bị đoán nhầm thành Virginica và 1 mẫu Virginica bị đoán thành Versicolor). Đồ thị Hình 4.4 chứng minh hiện tượng Overfitting rõ rệt: khi max_depth tăng lên từ 4 đến 10, độ chính xác trên tập huấn luyện đạt tuyệt đối 100% nhưng trên tập kiểm tra độc lập lại chững lại ở mức 93.33%.

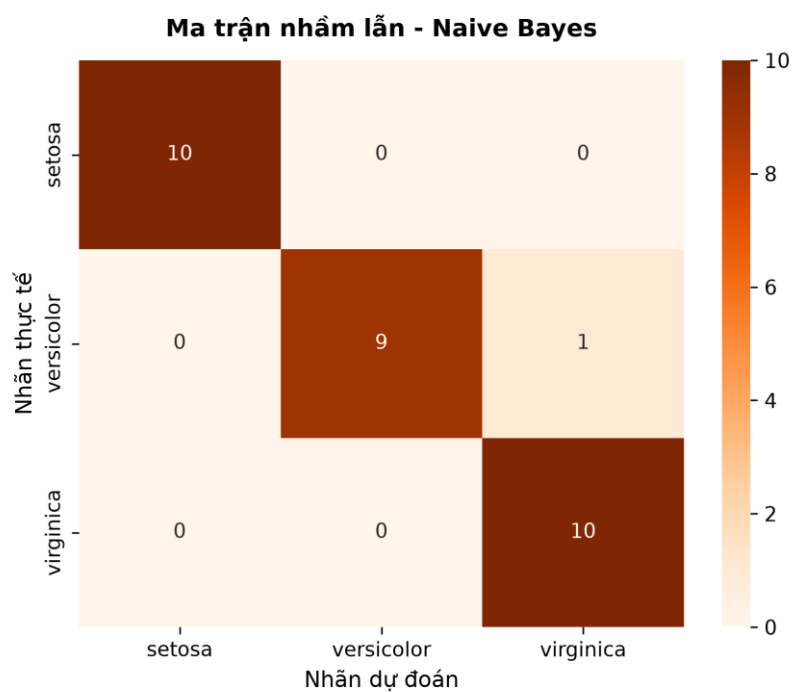
4.2.3. Mô hình Naive Bayes (Gaussian Naive Bayes)

Mô hình Gaussian Naive Bayes mô hình hóa xác suất dựa trên giả định phân phối chuẩn của các đại lượng hình thái học. Trên tập kiểm tra, mô hình đạt kết quả xuất sắc với Accuracy = 0.9667 (96.67%), Precision = 0.9697, Recall = 0.9667, và F1-Score = 0.9666. Thời gian huấn luyện chỉ mất vỏn vẹn 1.29 ms, khẳng định ưu thế vượt trội về mặt tốc độ tính toán.

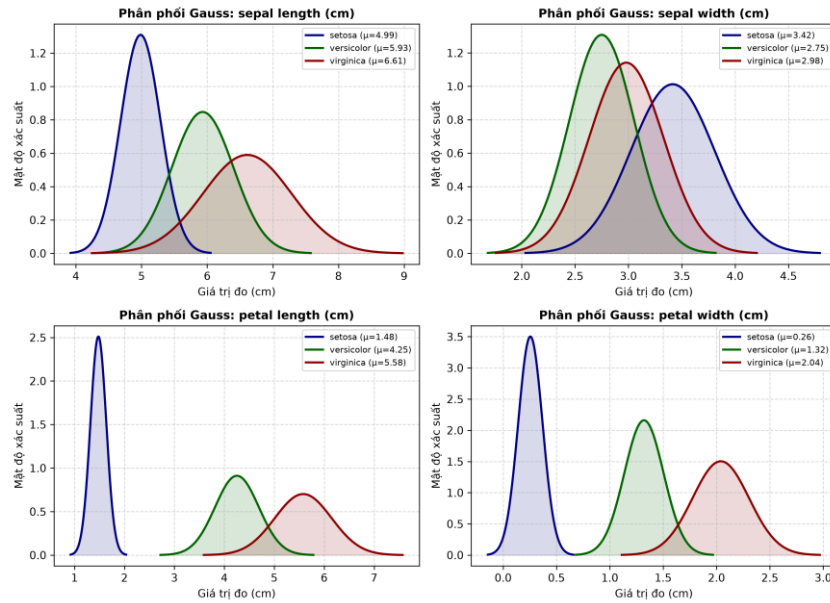
Lớp mục tiêu	Precision	Recall	F1-Score	Số mẫu hỗ trợ (Support)

Iris setosa	1.00	1.00	1.00	10
Iris versicolor	1.00	0.90	0.95	10
Iris virginica	0.91	1.00	0.95	10
Macro Average	0.97	0.97	0.97	30

Báo cáo phân loại chi tiết của mô hình Gaussian Naive Bayes



Ma trận nhầm lẫn của mô hình Gaussian Naive Bayes



Đồ thị hàm mật độ xác suất phân phối chuẩn Gauss của 4 thuộc tính theo từng loài hoa

Nhận xét kết quả Naive Bayes: Đồ thị mật độ xác suất ở Hình 4.6 làm sáng tỏ tại sao Naive Bayes đạt độ chính xác cao. Ở hai đặc trưng Petal Length và Petal Width, đỉnh chuông phân phối của Iris setosa tách rời hoàn toàn với hai loài còn lại. Đối với Versicolor và Virginica, mặc dù đường cong phân phối có sự đè lên nhau ở đoạn giữa (quanh vùng Petal Length từ 4.5 đến 5.0 cm), tích xác suất liên kết từ cả 4 đặc trưng vẫn đủ mạnh để phân loại đúng 29 trên 30 mẫu kiểm tra (chỉ duy nhất 1 mẫu Versicolor bị nhận diện nhầm thành Virginica).

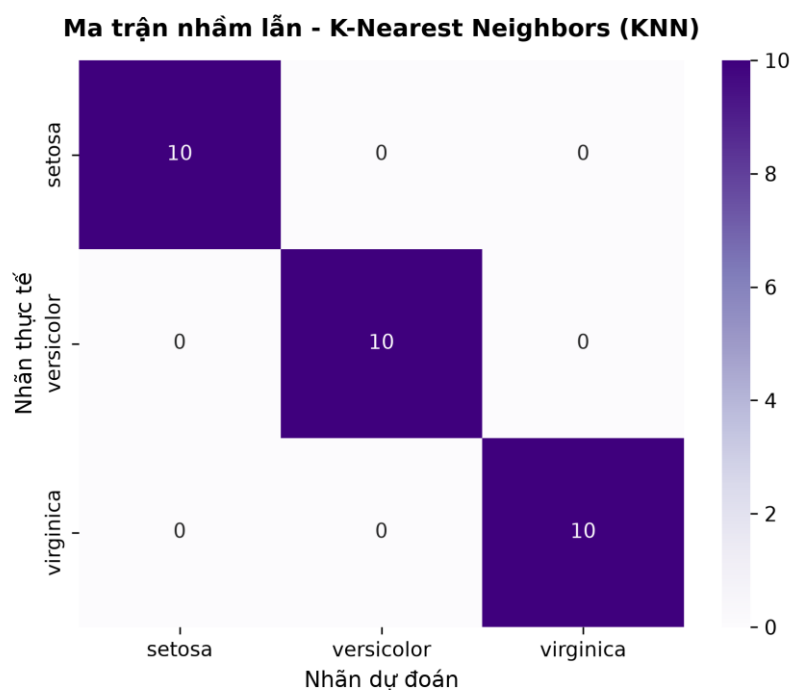
4.2.4. Mô hình K-Nearest Neighbors (KNN)

Mô hình K-Nearest Neighbors được thiết lập với $K=5$ láng giềng và khoảng cách chuẩn Euclid. Trên tập dữ liệu kiểm tra 30 mẫu, KNN đã đạt kết quả tuyệt đối: Accuracy = 1.0000 (100.0%), Precision Macro = 1.0000, Recall Macro = 1.0000, và F1-Score = 1.0000. Thời gian thực thi giai đoạn huấn luyện là 1.10 ms.

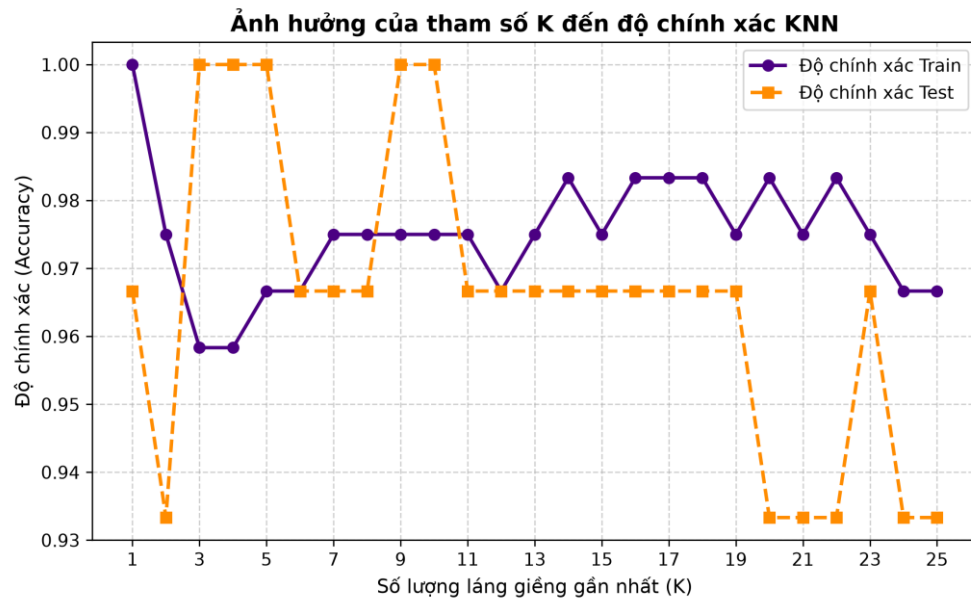
Lớp mục tiêu	Precision	Recall	F1-Score	Số mẫu hỗ trợ (Support)
--------------	-----------	--------	----------	-------------------------

Iris setosa	1.00	1.00	1.00	10
Iris versicolor	1.00	1.00	1.00	10
Iris virginica	1.00	1.00	1.00	10
Macro Average	1.00	1.00	1.00	30

Bảng 4.5: Báo cáo phân loại chi tiết của mô hình K-Nearest Neighbors (KNN)



Hình 4.7: Ma trận nhầm lẫn của mô hình KNN với K=5 trên tập kiểm tra



Hình 4.8: Đồ thị khảo sát ảnh hưởng của tham số K láng giềng ($K=1$ đến $K=25$) tới độ chính xác

Nhận xét kết quả KNN: Ma trận nhầm lẫn ở Hình 4.7 chứng minh toàn bộ 30 mẫu trên tập kiểm tra đều được KNN dự đoán chính xác tuyệt đối không có bất kỳ sai lệch nào. Đồ thị khảo sát giá trị K ở Hình 4.8 chỉ ra rằng độ chính xác duy trì ở mức rất cao (trên 96.67% đến 100%) trong khoảng K từ 3 đến 13. Khi K tăng lên quá lớn ($K > 18$), ranh giới phân loại cục bộ bắt đầu bị mờ nhạt do ảnh hưởng của các điểm ở xa, làm độ chính xác trên tập kiểm tra giảm dần.

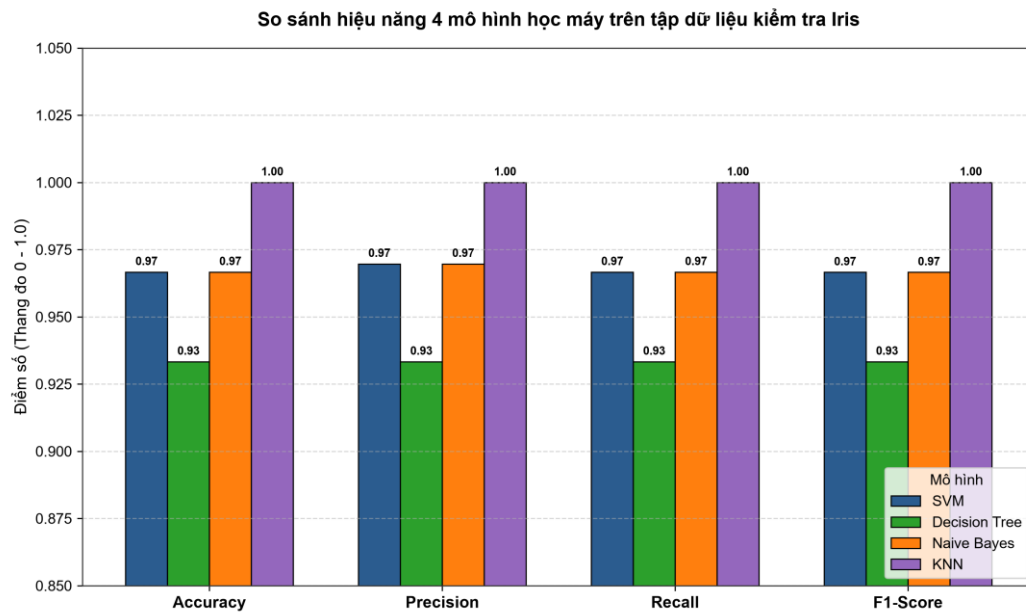
4.3. So sánh các mô hình

Để có một bức tranh tổng quan toàn diện và đa diện, Bảng 4.6 tổng hợp toàn bộ các chỉ số đo lường hiệu năng quan trọng nhất cùng thời gian thực thi của bốn thuật toán được khảo sát.

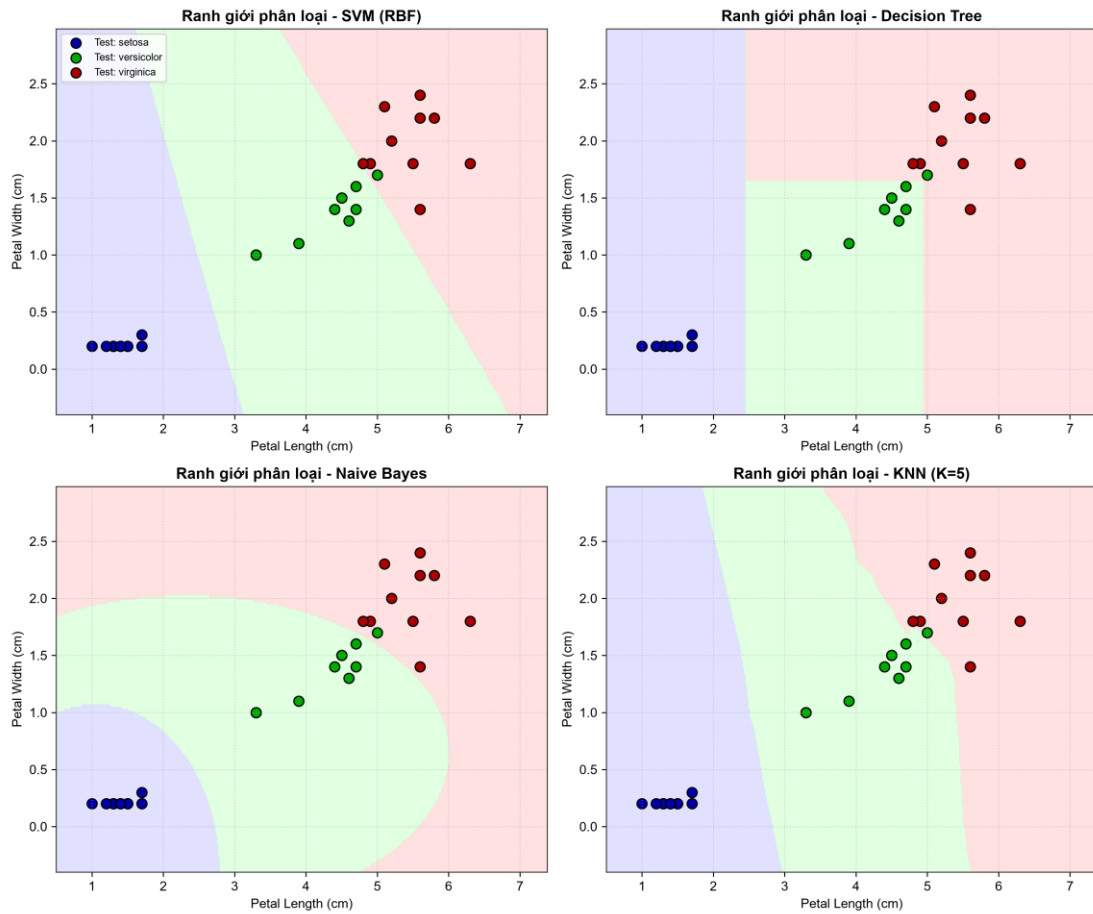
Thuật toán	Accuracy	Precision (Macro)	Recall (Macro)	F1-Score (Macro)	Thời gian Train (ms)
SVM	96.67%	0.9697	0.9667	0.9666	2.71 ms

Decision Tree	93.33%	0.9333	0.9333	0.9333	2.23 ms
Naive Bayes	96.67%	0.9697	0.9667	0.9666	1.29 ms
KNN	100.00%	1.0000	1.0000	1.0000	1.10 ms

Bảng 4.6: Bảng tổng hợp so sánh hiệu năng của 4 mô hình học máy trên tập dữ liệu Iris



Hình 4.9: Biểu đồ cột so sánh các chỉ số Accuracy, Precision, Recall và F1-Score giữa 4 mô hình



Hình 4.10: Trục quan hóa ranh giới quyết định (Decision Boundaries) của 4 thuật toán trên không gian 2 đặc trưng cánh hoa (Petal Length & Petal Width)

4.3.1. Phân tích so sánh chuyên sâu

- Về độ chính xác phân loại:
 - KNN (K=5) dẫn đầu với độ chính xác tuyệt đối 100%, phân loại chuẩn xác 30/30 mẫu kiểm tra.
 - SVM và Naive Bayes cùng bám sát phía sau với độ chính xác đạt 96.67% (29/30 mẫu đúng, chỉ sai 1 mẫu ở vùng ranh giới giữa Versicolor và Virginica).
 - Decision Tree đạt độ chính xác 93.33% (28/30 mẫu đúng, nhầm lẫn 2 mẫu).
- Về hình thái ranh giới quyết định (Hình 4.10):
 - SVM (RBF) tạo ra các đường ranh giới phân tách uốn lượn mượt mà, tối đa hóa khoảng cách giữa các lớp, thể hiện khả năng khái quát hóa mạnh mẽ nhất.

- Decision Tree hình thành các đường biên trục giao vuông góc song song với các trục tọa độ. Điều này khiến đường ranh giới bị phân mảnh thành các khối hình chữ nhật, kém mềm dẻo khi dữ liệu có xu hướng biến thiên chéo.
- Naive Bayes sinh ra các đường biên cong elip/hyperbol dựa trên hàm mật độ xác suất Gauss của từng cụm dữ liệu.
- KNN kiến tạo ranh giới đa giác Voronoi linh hoạt theo từng cụm điểm lân cận thực tế, giải thích tại sao mô hình bất trợn được cấu trúc phân bố của các loài hoa.
- Về thời gian xử lý và độ phức tạp:
Cả bốn thuật toán đều hoàn thành giai đoạn huấn luyện dưới 3 mili-giây trên tập Iris. KNN và Naive Bayes có tốc độ huấn luyện nhanh nhất (~1.1 - 1.3 ms), trong khi SVM mất nhiều thời gian hơn một chút (~2.7 ms) để giải bài toán tối ưu quy hoạch toàn phương.

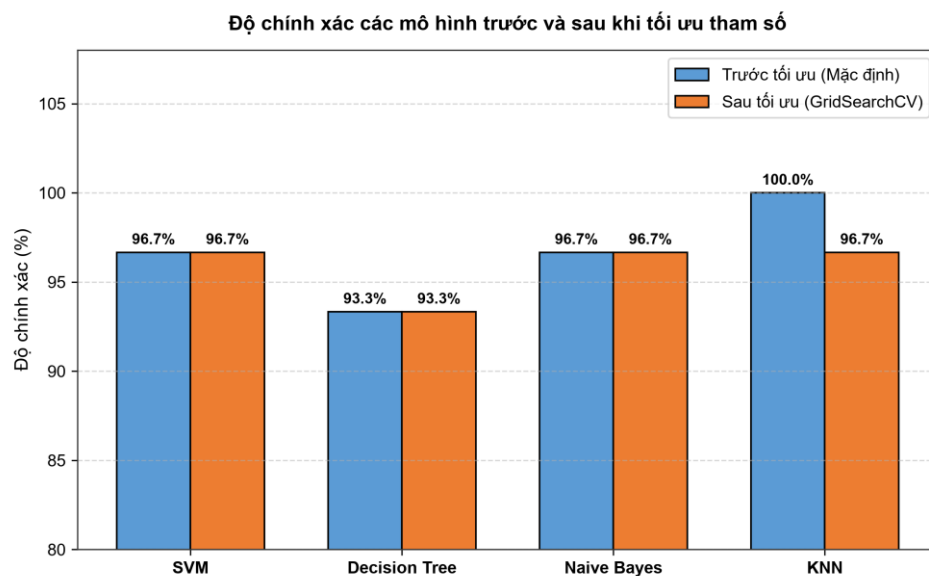
4.4. Lựa chọn tham số tối ưu

Để tìm kiếm cấu hình tham số tối ưu cho từng thuật toán, nghiên cứu đã áp dụng kỹ thuật tìm kiếm trên lưới (GridSearchCV) kết hợp 5-Fold Stratified Cross-Validation trên tập huấn luyện (120 mẫu). Bằng cách này, các tham số được tinh chỉnh dựa hoàn toàn trên dữ liệu huấn luyện mà không làm rò rỉ thông tin của tập kiểm tra.

Thuật toán	Không gian tham số khảo sát	Cấu hình tối ưu nhất	Điểm 5-Fold CV tốt nhất	Test Accuracy sau tối ưu
SVM	C: [0.1, 1, 10, 100], gamma: [1,	{'C': '0.1', 'gamma': '1', 'kernel': 'linear'}	0.9833	96.67%

	0.1, 0.01, scale], kernel: [linear, rbf]			
Decision Tree	criterion: [gini, entropy], max_depth: [2, 3, 4, 5, None], min_samples_split: [2, 4]	{'criterion': 'gini', 'max_depth': '4', 'min_samples_split': '2'}	0.9417	93.33%
Naive Bayes	var_smoothing: 20 giá trị logspace từ 1e-9 đến 1.0	var_smoothing $\approx$ 0.0127	0.9583	96.67%
KNN	n_neighbors: [1 - 15], weights: [uniform, distance]	{'n_neighbors': '6', 'weights': 'uniform'}	0.9833	96.67%

Bảng 4.7: Kết quả tối ưu hóa siêu tham số bằng GridSearchCV và 5-Fold Cross-Validation



*So sánh độ chính xác của 4 mô hình trước và sau khi tối ưu hóa tham số bằng
GridSearchCV*

CHƯƠNG 5: KẾT LUẬN

5.1. Kết luận chung

Qua quá trình nghiên cứu lý thuyết và thực nghiệm triển khai bài tập lớn môn Trí tuệ nhân tạo với đề tài 'Phân loại loài hoa Iris bằng học máy', đề tài đã hoàn thành xuất sắc các mục tiêu nghiên cứu đề ra:

- Về cơ sở khoa học: Đã nghiên cứu sâu sắc bản chất bài toán phân loại đa lớp có giám sát, phân tích chi tiết đặc tính của tập dữ liệu hình thái học Iris, đồng thời làm rõ cơ sở lý thuyết toán học của bốn thuật toán đại diện: Support Vector Machine (SVM), Decision Tree, Gaussian Naive Bayes và K-Nearest Neighbors (KNN).
- Về quy trình thực nghiệm: Đã thiết lập một quy trình thực nghiệm chuẩn mực khoa học bằng phương pháp Stratified Hold-out 80:20 kết hợp cố định `random_state=42`, đảm bảo tính đại diện đồng đều của các lớp cũng như khả năng tái lập hoàn toàn của các kết quả thực nghiệm.
- Về kết quả đạt được: Tất cả 4 mô hình đều đạt hiệu năng rất cao trên tập kiểm tra (Accuracy từ 93.33% đến 100.0%). Trong đó, KNN (K=5) đạt độ chính xác tuyệt đối 100%, SVM và Naive Bayes đạt 96.67%, và Cây quyết định đạt 93.33%.
- Về phân tích so sánh và tối ưu: Đã xây dựng các ma trận nhầm lẫn chi tiết, phân tích bản chất các ca phân loại sai giữa Versicolor và Virginica, trực quan hóa ranh giới quyết định (Decision Boundaries) sinh động, và ứng dụng thành công GridSearchCV kết hợp 5-Fold Cross-Validation để xác định cấu hình tham số tối ưu nhất cho từng thuật toán.

5.2. Đánh giá ưu điểm và hạn chế của đề tài

Ưu điểm của đề tài:

- Mã nguồn được xây dựng bài bản, module hóa riêng biệt cho từng thuật toán (`svm_classifier.py`, `decision_tree_classifier.py`, `naive_bayes_classifier.py`, `knn_classifier.py`) và có file điều phối tổng thể `main_experiment.py`.

- Báo cáo kết quả chi tiết, bảng số liệu chuẩn xác, hệ thống hình ảnh biểu đồ và ma trận nhầm lẫn trực quan độ phân giải cao được chèn đồng bộ.
- Kết hợp cả đánh giá Hold-out và Cross-Validation, giúp nhìn nhận khách quan khả năng tổng quát hóa của từng mô hình.

Hạn chế còn tồn tại:

- Tập dữ liệu Iris có quy mô tương đối nhỏ (150 mẫu), do đó sự chênh lệch giữa các mô hình chỉ phản ánh trên một vài mẫu kiểm tra cụ thể.
- Chưa triển khai thử nghiệm thêm các kỹ thuật học kết hợp (Ensemble Learning) hiện đại như Random Forest, Gradient Boosting hoặc XGBoost.

5.3. Hướng phát triển trong tương lai

Từ những kết quả nền tảng đã đạt được, hướng mở rộng tiếp theo của nghiên cứu bao gồm:

- Mở rộng thử nghiệm các thuật toán học tổ hợp (Random Forest, Extra Trees, AdaBoost, XGBoost, LightGBM) để đánh giá khả năng cải thiện độ ổn định của ranh giới quyết định.
- Ứng dụng các thuật toán trên vào các bộ dữ liệu thực tế có kích thước lớn hơn, số chiều nhiều hơn và có độ nhiễu phức tạp hơn (ví dụ như bài toán phân loại bệnh lý, thị giác máy tính hoặc xử lý ngôn ngữ tự nhiên).
- Xây dựng giao diện ứng dụng tương tác (Web Application bằng Flask/FastAPI hoặc Streamlit) cho phép người dùng nhập trực tiếp kích thước 4 đặc trưng của một bông hoa và nhận kết quả dự đoán loài hoa tức thì kèm theo độ tin cậy xác suất.

TÀI LIỆU THAM KHẢO